CRYPTORESEARCH · READ FROM THE CODE

# The Living System Atlas

A private research desk where models read and argue, code measures and keeps the books, and Eric decides. A visual litepaper and a map of the whole system at one source baseline, each part linked to the source that makes it true.

| 64 | 75 | 199 | 45 |
|---|---|---|---|
| components | connections | source links | claims checked |

**Source baseline 7057e06** · kuaya1/CryptoResearch · built November 3, 2026

The last repository receipt of the online desk's /healthz reads 5a8e75b at 2026-11-03 17:20 UTC (R0). Code has changed on main since then, so that receipt does not describe this baseline, and R0 must be repeated. This atlas does not read which revision is deployed now.

A static architectural and status map, not a live telemetry backend. It shows what the repository said when this page was built, and reads nothing from a desk or a scheduler while open.

This atlas describes code. It grants no authority, and it is not proof that code, deployment and operation match.

# Contents


I The Litepaper

01 A desk, not a bot

02 The owner's map

03 Identify: how a project enters

04 Research: how a report is made

05 Two agents, one argument

06 Challenge: the exit question

07 Authorize: the decision

08 Record: four books, four totals

09 Who decides what

10 Where it runs

11 How it changes

12 Not built, disabled, unproven

13 Stage 1.5: holds, proof and what is next

II The full map, in three pages

III Six guided paths

IV Component reference

V How to read this atlas, and what it could not establish

01

# A desk, not a bot

CryptoResearch is a private research and paper-trading desk. Models read, argue and write. Code measures, sets risk ceilings and keeps the books. Eric decides, himself or through a paper-only mandate he can arm. No path reaches live capital.

Every project on the desk goes through the same five handoffs: it is identified, researched under a frozen prompt and graded by the desk itself, challenged on how it could fail and whether a position could get out, authorized up to a ceiling, and recorded.

The records are plain files, written through a locked and verified operation. There is no hidden database, and nothing on the desk moves real money.

This atlas describes the code at its baseline. Whether a part is switched on, deployed or proven in operation are separate facts, given separately, and UNKNOWN where the repository holds no evidence.



### Models read and argue

Claude, ChatGPT and Gemini research. A second Claude prosecutes. They write words, never amounts.



### Code measures and keeps books

Grades, sizing, exits, orders and stops are deterministic JavaScript with exact decimals.



### Eric decides

Every decision is his or a paper mandate's he armed; every switch that enables automation, and every merge, is his. Nothing reaches live capital.

**THE TECHNICAL DETAIL**

- The Constitution splits the system in two. A cognitive shell of language models does qualitative research and cross-examination. A deterministic core in JavaScript does sizing, accounting, order states and stops, with exact decimal arithmetic.
- Eric keeps merge authority, capital and the final decision. No loop is ever given those.

`docs/CONSTITUTION.md:29` `README.md`

02

# The owner's map

Five handoffs: Identify, Research, Challenge, Authorize, Record. Evidence services, the platform and engineering support all five.

Each stage answers one question and hands one thing to the next. Each record says where it came from. The full map, with all 64 parts, is the advanced view of the same five stages.

### 1 Identify

**Is this the asset we think it is?**

A link, a scout lead or an armed admission becomes one project with a verified CoinGecko identity and contract.

→ **A project with a stable identity**

### 2 Research

**What does the evidence say?**

One frozen prompt and evidence packet per run. Each model's report is kept as its own opinion; the desk computes a qualified grade, or withholds it.

→ **A graded report per model, never averaged**

### 3 Challenge

**How could it fail, and can a position get out?**

The Prosecutor argues the short case and writes the trade plan; exit quotes and contract reads measure what selling would cost. ChatGPT's challenge is advisory.

→ **A ticket with a measured exit**

### 4 Authorize

**Who allowed this risk, and how much at most?**

Code suggests a risk ceiling. Eric approves, halves, vetoes or passes; or, only if he armed it, the paper mandate grants on the Trader's book within his limits and expiry. No path authorizes live capital.

→ **A decision, or a mandate grant, with a ceiling**

### 5 Record

**What happened as a result?**

The facts behind each decision are frozen with it. What follows is kept apart: a benchmark position, a simulated order that may wait, be refused or fill, report outcomes, and Eric's real holdings.

→ **Four separate records, four separate totals**

**Evidence services**
Prices, fundamentals, contract flags, quotes and chain reads that every stage draws on.

**Platform**
Hosts, sign-in, the API, timers, record writes, files and backups.

**Engineering**
How the code changes: supervised, one task, Eric's merge. Not proof that code, deployment and operation match.

03

# Identify: how a project enters

Three admission branches: Eric confirms a candidate, his own exact link confirms itself, or automatic admission admits an eligible lead inside limits he armed.

The usual door is New research. Eric gives a link, a name or a screenshot, and the desk finds CoinGecko candidates. When his own CoinGecko link or exact contract resolves to exactly one listing, it confirms itself; a name search, a screenshot or several candidates wait for his choice.

Every morning a Grok sweep can bring a reading brief and token leads. A brief never starts anything. A lead is checked against CoinGecko and GoPlus, and is admitted by Eric's "Confirm and request research", or by automatic admission, which is off by default, expires, and is blocked while automation is paused.

### Eric confirms

He picks one of the CoinGecko candidates, or clicks "Confirm and request research" on the scout leads he saw.

**Manual**

### A single exact match confirms itself

Only when his own CoinGecko link or exact contract resolves to one unambiguous listing. Never a name search or a screenshot.

**Automatic, his input**

### Automatic admission

Eligible scout leads, up to his daily limit, until the expiry he set. Off by default; blocked while paused.

**Automatic, if armed**

On the map: New research · Grok Scout · Scout briefs · Scout leads · Automatic admission · Project records

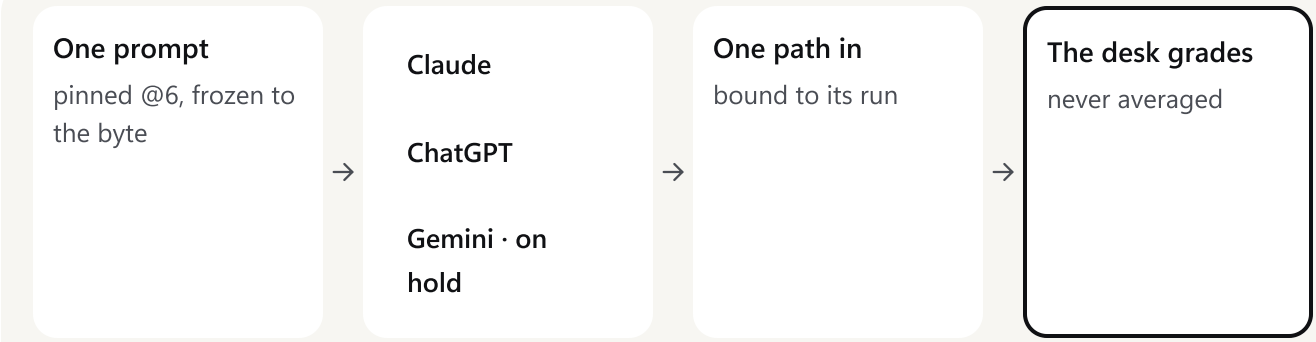
04

# Research: how a report is made

One prompt, frozen to the byte, is given to every model. The desk, not the model, calculates the grade.

A research run freezes the prompt and an evidence packet of dated prices, fundamentals and desk measurements. Each model gets the identical text.

A report can arrive from a paid API, from an assistant through the connector, or as a pasted answer. All three take the same path in, are bound to their run, and are graded by the same code. An unknown component withholds the grade; it is never filled in.




Each model’s report keeps its own grade. A missing component withholds the grade rather than guess it.

**THE TECHNICAL DETAIL**

- Prompt @6 asks every model three tests: the payment waterfall, holder cash flow per token against supply growth, and one falsifiable thesis.
- Evidence checks read meaning, not field presence, and only flag. They never change a score or a stored report.
- Grades from different models are shown side by side and never averaged.

On the map: Research runs · Pinned prompts · The connector · Report intake · Grading · Evidence checks · Compare & revisions

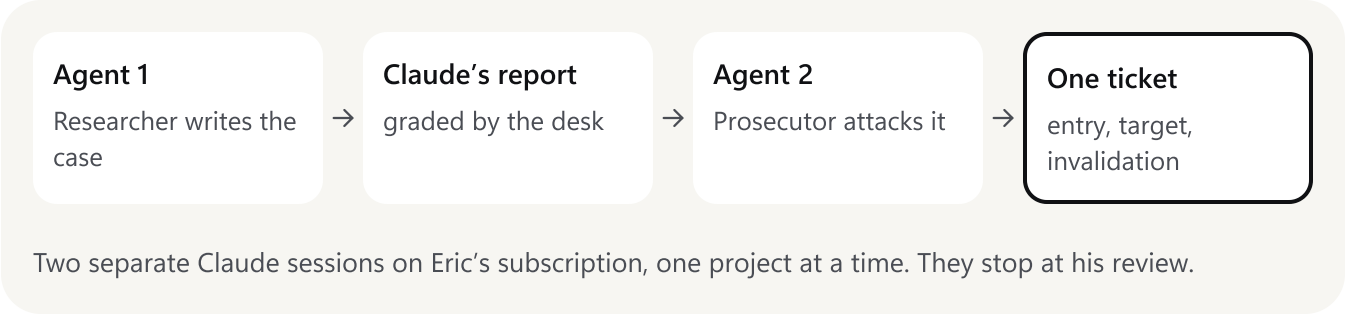
05

# Two agents, one argument

Agent 1 makes the case. Agent 2 tries to destroy it. Neither picks a dollar amount.

The pilot runs Claude twice, in separate sessions on Eric’s own subscription. The Researcher writes Claude’s report under prompt @6. The Prosecutor then attacks that report as a short-seller and writes one ticket: an entry zone, an upside target and an invalidation price.

ChatGPT may then challenge the ticket on four tests. Its verdict is shown in words and changes nothing.




Two separate Claude sessions on Eric’s subscription, one project at a time. They stop at his review.

**THE TECHNICAL DETAIL**

- The runner claims one project at a time and stops at Eric’s review.
- It is implemented and registered Disabled on the PC. No run is recorded: at R0 the task’s state was never_run, and the One Sheet says “Operating proof: none yet”.

On the map: Pilot runner · Agent 1 · Researcher · Agent 2 · Prosecutor · ChatGPT challenge

06

# Challenge: the exit question

Before any risk, the desk asks whether a position of that size could actually get out, and at what cost.

Forensic audits investigate how a project could fail: liquidity impairment, operational degradation, delivery inactivity or discontinuation. They are separate from research and change no grade.

The monitor quotes the exit at the position’s size from Jupiter or KyberSwap. A cost is complete only when fees and taxes are known; otherwise it reads “at least”. Every approval needs a reading under 72 hours old.

On the map: Forensic audits · Forensic monitor · Exit readings · Contract forensics

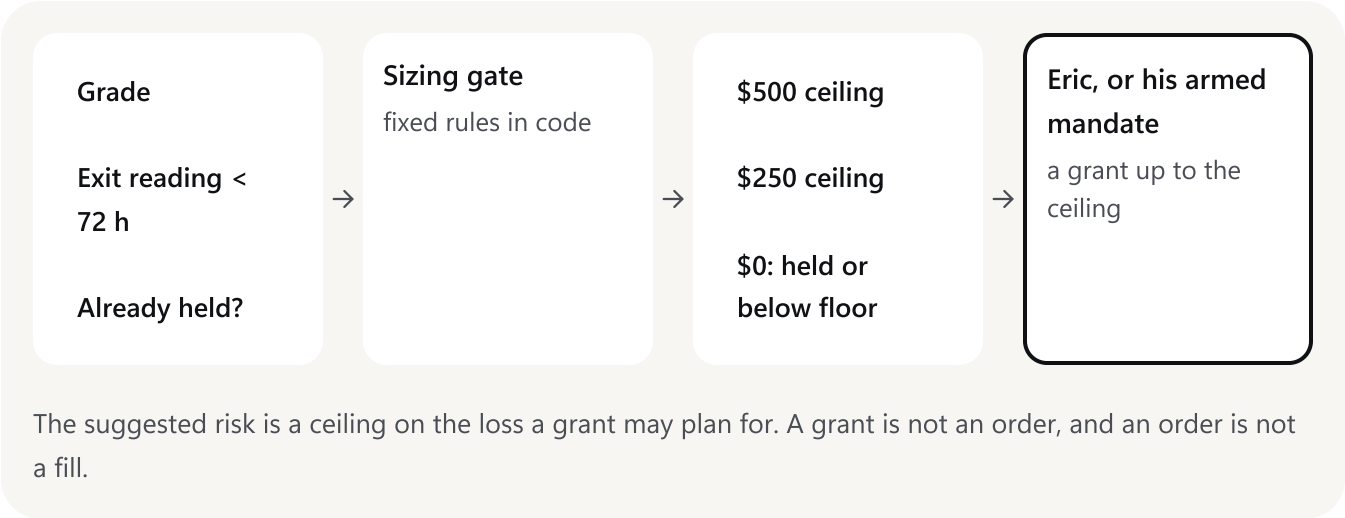
07

# Authorize: the decision

Code sets a risk ceiling. Eric authorizes up to it, or a paper mandate he armed does. The desk refuses a decision made on stale facts.

The sizing gate turns the grade, the exit reading and whether Eric already holds the asset into a suggested risk: $500, $250 or $0. That figure is a ceiling on the loss a grant may plan for. It is not a position size, an order or a fill.

On the PM Decision Card Eric approves, halves, vetoes or passes. The desk freezes what he decided on in the same write, and refuses if a newer report arrived, the trigger was breached, the exit reading expired or the card changed.




The suggested risk is a ceiling on the loss a grant may plan for. A grant is not an order, and an order is not a fill.

**THE TECHNICAL DETAIL**

- Delegated paper authority: the paper mandate can grant on pilot projects by grade, on the Trader's book only, inside an authority generation recorded when Eric arms it, expiring within 30 days. It is off by default; a pause revokes it and a resume does not re-arm it; any decision of his own ends it on that project. Its presence in the code authorizes nothing.
- Live-capital authority: none exists.

On the map: PM tickets · Sizing gate · Eric's decision · Paper mandate · Off switches

08

# Record: four books, four totals

What a decision stood on is frozen with it. What followed is kept in four records with four separate totals, never added together.

A grant is a permission up to a ceiling. In the PM benchmark it becomes a position at the signal price whose result is the granted dollars times the price move. In the Trader simulation it becomes one intent: the grant is the planned loss at the stop, the quantity and notional follow from the stop distance under the notional cap, and the order may be refused, wait, fill in part or fill. Report performance measures every eligible report against BTC, traded or not. Real holdings are Eric's own ledger.

### PM benchmark

Eric's own approve or halve, opened at the signal price. Its result is the granted dollars times the price move: an attribution of his decisions, not funded execution. Mandate grants never enter it.

### Trader simulation

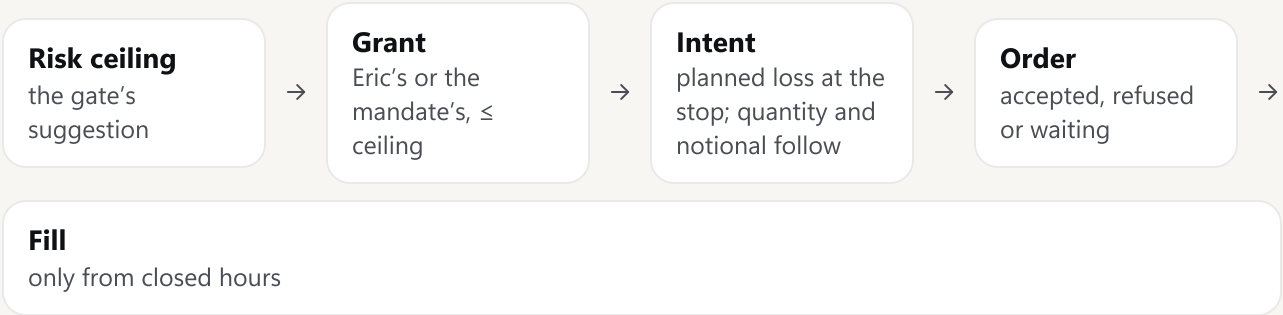
Funded paper execution: an intent sized from a planned loss at the stop, checked by the risk engine, sent to a simulated venue, filled only from closed hours, with fees. An order can wait, be refused or fill in part.

### Report performance

Every eligible report against BTC at 30–120 days, traded or not. Computed on read, nothing stored, no ranking; medians only from 20 matured reports.

### Real holdings

Eric's recorded holdings and cost, and the wallets he follows read only. Nothing here places an order.




Four records, four totals, never added together. The chain applies to the Trader simulation; the PM benchmark stops at the grant.

**THE TECHNICAL DETAIL**

- The Trader’s book stays off until Eric sets its funding, fee and rules.
- An order never times out to “lost”. It stays unknown until the venue is asked by its order ID.
- The PM benchmark shows no totals before 20 closed positions; report performance shows a median only from 20 matured reports. Grades stay each model’s own, qualified or withheld.
- Outcomes reach Eric only. Nothing feeds back into a grade, a rubric or the mandate.

**Checked in the code** a 20-report floor is stated (`report-outcomes.js:2`)

On the map: Decision basis · PM benchmark · Trader adapter · Risk engine · Trader simulation · Hourly candles · Report performance · Real holdings · Wallet sync

09

# Who decides what

Three kinds of authority, kept apart: Eric’s own, a paper mandate he can delegate, and live capital, which no path reaches.

Automation acts only inside what is enabled. A project stop halts admission, the pilot, mandate grants and openings for that project until he rearms it.

The desk-wide pause stops new automated dispatches. It deliberately does not stop the safety and record-keeping work that must finish what is already open.

## Manual authority

**Eric**

Approve, Halve, Veto or Pass on the PM Decision Card. Approve and Halve grant up to the suggested risk ceiling; the desk refuses a decision on stale facts.

## Delegated paper authority

**The paper mandate, only if Eric arms it**

Off by default. When armed, grants on pilot projects by grade, on the Trader’s book only, within daily and open-grant caps and an expiry at most 30 days ahead. A pause, a loop switched off or Eric’s own decision revokes it; a resume does not re-arm it.

## Live-capital authority

**None exists**

There is no live venue in the code. The Trader’s venue is a file. This atlas grants no authority of any kind.

### The desk-wide pause stops

- Research sent to a provider (the research job and tick)
- The pilot runner's claim, preparation and next agent
- Standing exit-quote approvals (exit_quotes_auto) and the policy's primary approval
- Automatic lead admission; a pausing write revokes its generation
- Paper mandate grants; the mandate is revoked as "paused", and a resume does not re-arm it

### It leaves running

- Market prices, fundamentals and screening
- Forensic monitoring of projects already opted in, and contract observations
- Wallet sync
- The Trader's book: settlement from closed candles, cancels of a mandate buy no longer covered, closes, and reconciliation of unknown orders
- Snapshots, review findings, backups and the offsite pull
- Eric's own decisions and record changes

`dashboard/core/workspace.js:129` `dashboard/core/jobs.js:107`
`dashboard/core/mandate-authority.js:165` `dashboard/core/pilot-claims.js:122`
`dashboard/core/lead-admission.js:149` `dashboard/core/trader-desk.js:610`

| Automation | Enabled by | Default in code | Stopped by |
|---|---|---|---|
| **Grok Scout** | Repository variable SCOUT_ENABLED | Off unless set | The variable |
| **Automatic admission** | automation.lead_admission, with an expiry | Off | Admission off · project stop · pause |
| **Paid API research** | A key, a model, prices, a budget | Not configured | Pause · model hold |
| **Scheduled research** | research.unattended_enabled | Off | Pause |
| **Pilot runner** | Eric enables the task and enrolls projects | Task Disabled | Project stop · pause |
| **Paper mandate** | autonomous_paper_loop and an authority ≤ 30 days | Off | Mandate off · project stop · pause · any decision of his |
| **Trader's Paper Book** | trader.book and trader.rules | Unset | Unfunding the book (a pause leaves settlement running) |
| **Exit quotes** | Monitoring per project, or exit_quotes_auto | Off | Monitoring off · pause stops the standing policy |
| **Contract forensics** | contracts.enabled and compute caps | Off | The setting |
| **Wallet sync** | ALCHEMY_API_KEY and a wallet he adds | No wallet | Removing the key or wallet |

On the map: Off switches · Paper mandate · Automatic admission · Eric's decision

10

# Where it runs

One codebase, two hosts, and files you can open.

The online desk runs on Render from main, with its own records on its own disk. The PC desk runs the same code locally; its hourly task, when enabled, pulls the only off-Render copy of the records.

Record changes are designed to take one locked operation: lock, stage, atomic move, verify, log. A save counts only when it reads back verified. Whether every writer uses it has not been audited for this atlas: some modules also write files directly.

On the map: Online desk · PC desk · Front door · Desk API · Job timers · Journaled writes · Workspace files · Backups

11

# How it changes

Supervised, one task at a time. The self-building machine was retired on purpose.

Each change is one task on one branch in Eric’s own Claude Code session, then the test suite, a pull request and Eric’s merge. The trader’s core files are on one protected list: a boundary test checks what calls them and what they reach, and a change to one is its own pull request that Eric reviews. The test cannot tell whether a change was intended; review does that.

Merged is not deployed, and deployed is not operating: each is read back on its own.

On the map: Supervised loop · The test check · Protected trader files · Deploy · Agent lane

12

# Not built, disabled, unproven

The honest edge of the system, in three kinds: what does not exist, what exists but is off by default, and what has no operating proof.

## Not built

The thing does not exist.

- **Ops-1, the Render Cron Job that runs the pilot**

  A specification and preflight script exist; the service is not created.

  `docs/engineering/OPS1_RENDER_CRON.md`

- **A real venue**

  None exists. The Trader's venue is a file.

  `dashboard/core/paper-trader.js:827`

## Built, off by default

In the code; does nothing until enabled.

- **The pilot runner's PC task**

  Registered Disabled; Eric enables it.

  `dashboard/scripts/pilot-task.ps1:209`

- **The paper mandate**

  automation.autonomous_paper_loop is false by default.

  `dashboard/core/workspace.js:142`

- **Automatic lead admission**

  Off by default, with an expiry when armed.

  `dashboard/core/workspace.js:152`

- **Standing exit-quote approvals**

  exit_quotes_auto is false by default.

  `dashboard/core/workspace.js:141`

- **Contract reads and the contract gate**

  contracts.enabled is false; the gate observes.

  `dashboard/core/workspace.js:193`

- ◯ **Real records in the Explorer**

  One project only, by Eric's confirmation from his own terminal; this work sent none.

  `dashboard/core/explorer.js:596`

- ◯ **Gemini on the online desk**

  On hold; nothing new is assigned to it.

  `dashboard/hosted.js:61`

## No operating proof

No receipt in the repository shows it working.

- ◌ **An unattended pilot run**

  pilot-all never_run at R0; B0 and later gates unstarted.

  `docs/engineering/OPERATING_PROOF_R0_2026-11-03.md:33`

- ◌ **The PC's hourly run and offsite copy**

  Hourly left temporarily Disabled at R0; no new offsite run claimed.

  `docs/engineering/OPERATING_PROOF_R0_2026-11-03.md:89`

- ◌ **A mandate grant, a funded Trader's book, a simulated fill**

  No receipt in the repository.

  `docs/ONE-SHEET.md:74`

- ◌ **Each desk's current settings**

  Not read by this atlas: UNKNOWN.

  `dashboard/core/workspace.js:139`

## Still open

Eric's own decisions.

- ☐ **Eric's open questions**

  The Law 1 credential: B1 closed only its environment path. The buy-and-hold baseline is deferred to Stage 2 by Eric's ruling. See chapter 13.

  `docs/ONE-SHEET.md:76`

On the map: Ops-1 runner · Pilot runner · Paper mandate · Trader simulation · Research Explorer

13

# Stage 1.5: holds, proof and what is next

Seven planning items in three classes that must never be read as one another: held on purpose, waiting for a real run, and code built but not yet run on real records. Operating proof: none yet.

All the Stage 1.5 code slices come first. The live proof, R0 then B0 to B3, runs once afterward (Milestone D), so that main and the deployed revision stay frozen while it runs.

A static architectural and status map, not a live telemetry backend. It shows what the repository said when this page was built, and reads nothing from a desk or a scheduler while open.

## Intentional hold by design

Eric chose this. It is not an engineering failure or a gap to close.

3 **Real trading venue**

None exists. The Trader's venue is a file, and Stage 1.5 stays paper-only. Stage 2 needs Eric's separate authorization.

`dashboard/core/paper-trader.js:827`

6 **Gemini on the online desk**

On hold, so nothing new is assigned to it. It stays held until the B0 to B3 evidence and the Ops-1 move are complete.

`dashboard/hosted.js:61`

## Pending operating proof

The code or specification exists. No real run is recorded, and passing tests are not one.

1 **Ops-1: the independent runner**

The Render Cron Job is specified, and its service is not created. Moving to it is the last step of Milestone D.

`docs/engineering/OPS1_RENDER_CRON.md:3`

2 **B0 to B3: the operating proof**

prove-autonomy (#169, #171) now reads the evidence for each step, without running anything. No real B0, B1, B2, B3 or cutover evidence exists yet, and R0 must be repeated.

`docs/engineering/AUTONOMY_PROOF_CLI.md`
`dashboard/core/autonomy-proof.js`

## Built in code

On main once merged. This says the code exists, not that it has run on real records. Whatever stays open is named.

4 **GoPlus in research packets**

Active in code (C2): new research and ticket packets carry GoPlus's flags from the saved screen. An unanswered check stays unknown, a missing, old or failed screen says so, and nothing is labeled safe. No real run has used it yet.

`dashboard/core/screening.js:100`
`dashboard/core/screening.js:914`
`dashboard/core/research.js:814`
`dashboard/core/tickets.js:719`

---

5 **Real records in the Explorer**

Built in code (C3): one project goes only by its preview's hash, from Eric's own terminal, and any change since the preview sends nothing. The whole desk stays refused. This work sent no real record.

`dashboard/core/explorer.js:565`
`dashboard/core/explorer.js:596`
`dashboard/cli.js:1322`

---

7 **Owner questions and protected correctness**

One still open, with no slice left: the Law 1 credential. B1 kept GitHub credentials out of child environments, but the stored gh login still reaches admin, so the law stays procedural until Eric separates it. C1 built the fee accounting (gross, fees, net and signal against fill per position). The buy-and-hold baseline is not built; Eric deferred it to Stage 2. A5 is closed in code (B3): a paper record that does not read is an INTEGRITY refusal, never a shorter book. The Constitution's Invariant 3 now names all nine protected files (B2).

`docs/ONE-SHEET.md:76`
`docs/CONSTITUTION.md:50`
`dashboard/core/child-env.js:95`
`dashboard/core/paper.js:45`
`dashboard/core/grade-outcome-view.js:132`

The classes and each item's class are Eric's rulings of 2026-11-03, in his Stage 1.5 directive and his orders for these pull requests; his order for C2 moved items 4 and 7 to built in code, and C3 moved item 5, the last

scheduled slice. Neither is a file in this repository, so the build checks the facts cited here but cannot check the rulings.

## Where each part runs

| Host | What runs there |
|---|---|
| **GitHub Actions** | The morning scout, while Eric's switch is on, and the test check on every pull request. `docs/ONE-SHEET.md:33` |
| **Render desk** | The web service and its 30-minute timer, and the 2-minute research tick. `docs/ONE-SHEET.md:34` |
| **Runner host** | The pilot, one at a time: the PC task during B0 to B3, then the Ops-1 Render Cron Job once created. `docs/ONE-SHEET.md:35` |
| **PC offsite backup** | The hourly task: the only copy of the records off Render. The desk runs without it. `docs/ONE-SHEET.md:36` |

## The sheet's boxes

No document in this repository names a set of six boxes, so this atlas does not name one. The sheet draws the boxes below. The map's six rows are the five handoffs plus evidence services.

1. Scout lead
2. Admission
3. Agent 1 research: one report per model, separate grades
4. Agent 2 prosecution: conviction and typed spread
5. Ticket receipt: create-only, frozen packet
6. Decision line + decision basis: one operation
7. Intent: one attempt per decision
8. Order: reserved, sent, acked
9. Causal fills, stop, target, expiry
10. Trader Sim outcome: fee-bearing
11. Research vs BTC: every eligible report
12. PM Cohort benchmark
13. Eric's review

**Over the whole path:** Stop: pause, mandate off, admission off, project stop · Liveness: poll sighting, run outcome, heartbeat

## Where this status comes from

| Pull request | Merge commit | What it did |
|---|---|---|
| `#158` | `34089b9` | N1: an unreadable digest ledger never vetoes a stop or Undo |

| Pull request | Merge commit | What it did |
|---|---|---|
| #159 | 1230170 | N3: the reconcile check binds to the attempt Eric checked |
| #160 | af03923 | N2: the local runner's import checks the stop inside its save |
| #161 | e3e017d | N4 and N7: only the journal snapshot the book validated |
| #162 | 6e2db98 | The autonomy completion blueprint, Q1 to Q5 of record |
| #163 | 94cede6 | The runner: unattended launch, revision guard, honest heartbeat |
| #164 | 21a11fa | Jobs: impaired status for partial failures |
| #165 | fbfb640 | The sheet's host legend and the /healthz deploy rule |
| #166 | 5a8e75b | Ops-1 specification and read-only preflight |
| #167 | 363b536 | The R0 receipt: the dedicated PC runner |
| #168 | eb4e0b4 | This atlas |
| #169 | 1909597 | prove-autonomy: the read-only evidence foundation |
| #170 | Not merged | Closed unmerged. Its checks were ported into #171. |
| #171 | 9f6d9aa | prove-autonomy: readers for B1, B2, B3 and cutover |

The sheet's merged list ends at #166. #167 to #171 are checked against git history, not the sheet.

`docs/ONE-SHEET.md:74` `docs/engineering/AUTONOMY_PROOF_CLI.md`

# II · The full map, 1 of 3 Evidence services, Identify, Research




● Implemented · on by default ○ Implemented · off until enabled ◌ Not created — Retired

Eric acts Automated Both ✦ AI agent

Arrows that leave the page continue on the other map page. Every connection, with what passes along it, is listed in the component reference.

# II · The full map, 2 of 3 Challenge, Authorize, Record

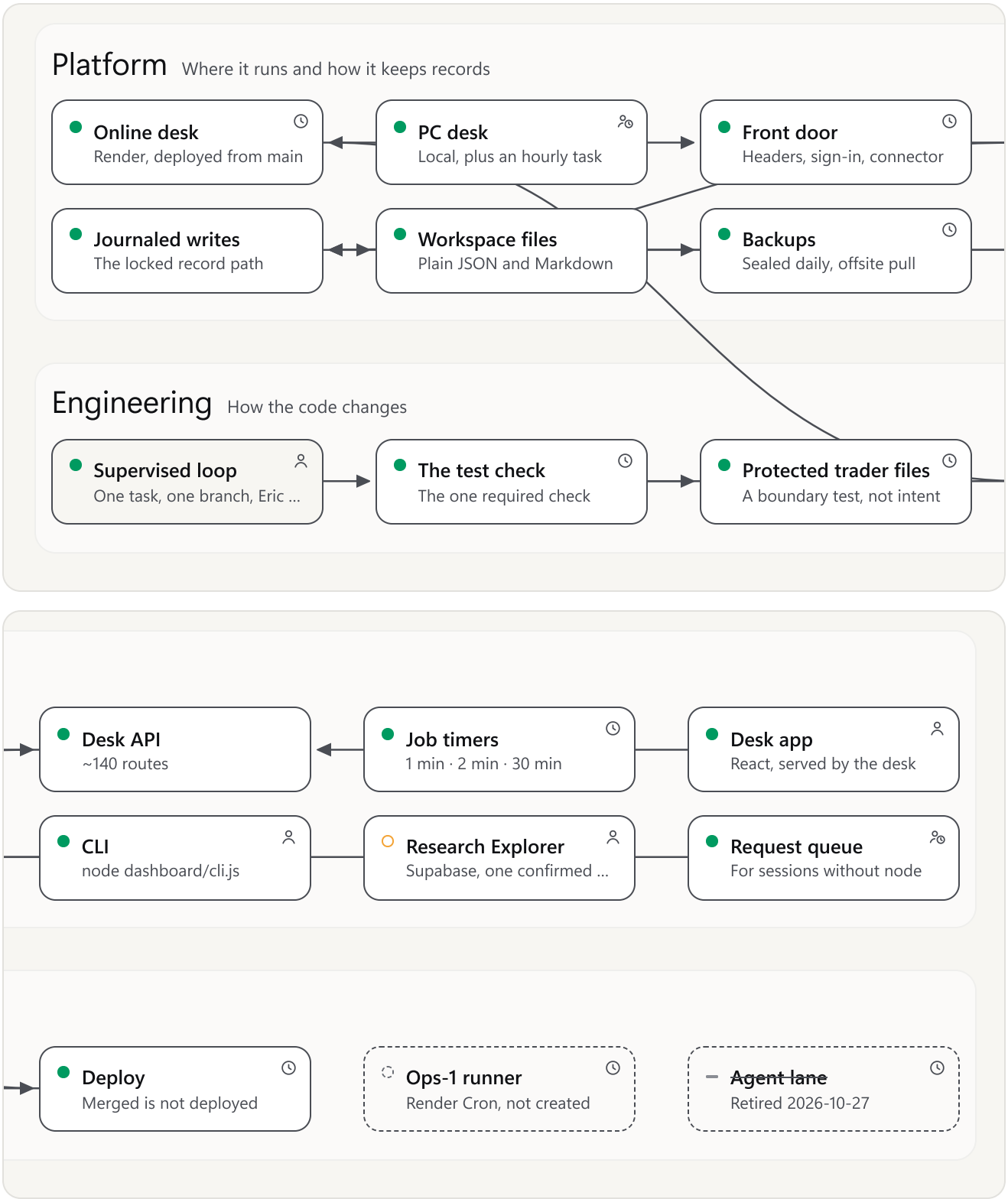



● Implemented · on by default ○ Implemented · off until enabled ◌ Not created — Retired

Eric acts · Automated · Both · ✦ AI agent

Arrows that leave the page continue on the other map page. Every connection, with what passes along it, is listed in the component reference.

# II · The full map, 3 of 3 Platform and Engineering




Implemented · on by default · Implemented · off until enabled · Not created · Retired

Eric acts · Automated · Both · ✦ AI agent

Every connection, with what passes along it, is listed in the component reference.

# III · Six guided paths

## A link becomes a decision

The main path, from Eric pasting a link to the PM Decision Card.

1. **New research** Eric pastes a link, a name or a screenshot.
2. **CoinGecko** The desk searches CoinGecko for candidates. Eric confirms one, unless his own exact link matched only one.
3. **Project records** A project folder is opened with its verified identity.
4. **Research runs** A research run freezes one prompt and one evidence packet.
5. **Pinned prompts** The prompt is the pinned @6 template, checked byte for byte.
6. **Agent 1 · Researcher** Claude researches and writes the report.
7. **Report intake** The report comes in one way and is bound to its run.
8. **Grading** The desk computes the grade itself.
9. **PM tickets** A ticket opens on Claude's report with a frozen packet.
10. **Agent 2 · Prosecutor** Agent 2 argues the short case and writes the trade plan.
11. **Sizing gate** Code, not a model, sets the suggested risk ceiling.
12. **Eric's decision** Eric approves, halves, vetoes or passes. A grant is a permission, not an order.

## The pilot, two agents

How Agent 1 and Agent 2 run on Eric's Claude subscription, one at a time.

1. **Pilot runner** The runner asks the desk's queue what is next and claims one project.
2. **Research runs** The desk prepares the frozen prompt; no paid API is used.
3. **Agent 1 · Researcher** Agent 1 runs in its own Claude session with web tools only.
4. **Report intake** The answer is handed back and saved as Claude's report.
5. **PM tickets** The queue moves the project to the ticket step.
6. **Agent 2 · Prosecutor** Agent 2 runs in a separate session and saves the ticket.
7. **Eric's decision** The pilot stops here. The decision is Eric's, or the paper mandate's if he armed it.

## Scout to research

From the morning Grok sweep to an admitted project.

1. **Grok Scout** At 11:30 UTC, GitHub Actions runs the pinned scout prompt.
2. **xAI Grok** Grok searches the web, one call per lane.
3. **Scout briefs** The brief is stored for Eric to read. It starts nothing.
4. **Scout leads** Token leads are checked against CoinGecko and GoPlus.
5. **Automatic admission** If Eric armed it, and it has not expired or been paused, eligible leads are admitted inside his caps.
6. **Project records** Otherwise only his click admits a lead.

## Can it get out?

The exit reading that every approval needs.

1. **Forensic monitor** For an opted-in project, the monitor sees a quote is due.
2. **Exit readings** It sizes the exit from the stored price, in exact arithmetic.
3. **Jupiter · KyberSwap** Jupiter or KyberSwap answer with a quote. Nothing is swapped.
4. **PM tickets** The reading is frozen into the ticket's packet.
5. **Sizing gate** The gate sizes within the exit-cost limit.
6. **Eric's decision** Approve is refused once the newest reading is over 72 hours old.

## A decision on paper

From Eric's approval to fills in the simulated book.

1. **Eric's decision** Eric approves or halves: a grant up to the ceiling.
2. **Decision basis** What he decided on is frozen in the same write.
3. **PM benchmark** The PM benchmark opens a position at the signal price (Eric's grants only).
4. **Trader adapter** The adapter turns the grant into one intent: the grant is the planned loss at the stop.
5. **Risk engine** The risk engine checks every limit, or refuses.
6. **Trader simulation** The simulated venue accepts the order; it may wait, fill in part or fill.
7. **Hourly candles** Only closed hours after the send can fill it, stop it or hit its target.
8. **Report performance** Results reach Eric only. Nothing feeds back into a grade.

# A code change

How the system itself changes.

1. **Supervised loop** Eric names one task; Claude works on its own branch.
2. **The test check** The test suite runs on the pull request.
3. **Protected trader files** The trader boundary test guards the protected files.
4. **Deploy** Eric merges; Render deploys main.
5. **Online desk** Deployed only when the health check shows the commit.

# IV · Component reference

**Unless a part says otherwise**

Implemented: yes, at 7057e06.

Deployed: Online desk: UNKNOWN at this baseline; last receipt 5a8e75b (R0, 2026-11-03 17:20 UTC). PC desk: UNKNOWN.

Enabled gives the default in code; what each desk is set to now is not read, so it is UNKNOWN.

# Evidence services Outside sources the desk reads

## CoinGecko Outside service · Evidence services

Implemented · on by default · On request

The desk's one source of prices and token identity. Every CoinGecko caller shares one rate-limit gate.

- Markets every minute for tracked coins.
- Identity search for new research.
- Contract lookups for scout leads.
- Works without a key; an optional demo key is sent as a header.

**Implemented:** Outside service; the desk's client code is at 7057e06

**Enabled:** On by default; each desk's setting UNKNOWN

**Deployed:** Not ours to deploy

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Market prices (prices)

**Receives from** New research (searches candidates) · Scout leads (checks the listing)

`dashboard/core/market.js:243`
`dashboard/core/market.js:305`
`dashboard/core/workspace.js:37`

## DefiLlama Outside service · Evidence services

Implemented · on by default · On request

Thirty-day fees, revenue and holders' revenue, read only for projects bound to a DefiLlama slug with a source link.

**Implemented:** Outside service; the desk's client code is at 7057e06

**Enabled:** On by default; each desk's setting UNKNOWN

**Deployed:** Not ours to deploy

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Fundamentals (fees, revenue)

`dashboard/core/fundamentals.js:23`
`dashboard/core/fundamentals.js:463`

## GoPlus Outside service · Evidence services

Implemented · on by default · On request

Mint, proxy, blacklist and tax flags for each verified contract. Facts only, never a verdict.

**Implemented:** Outside service; the desk's client code is at 7057e06

**Enabled:** On by default; each desk's setting UNKNOWN

**Deployed:** Not ours to deploy

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Contract screening (flags)

**Receives from** Scout leads (checks the contract)

`dashboard/core/screening.js:18`
`dashboard/core/screening.js:576`

## Model APIs Outside service · Evidence services

Implemented · off until enabled · On request

Claude, ChatGPT and Gemini, called directly only when a model has a key, a model name, prices and a budget. Otherwise research goes through the assistants or a pasted answer.

- Claude: api.anthropic.com Messages, streamed, with web search.
- ChatGPT: api.openai.com Responses, run in the background.
- Gemini: the Interactions API, run in the background; on hold on the online desk.

**Implemented:** Outside service; the desk's client code is at 7057e06

**Enabled:** Off by default; each desk's setting UNKNOWN

**Deployed:** Not ours to deploy

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** An API key in the environment or secrets file · providers.<model>.model and prices in Settings · A research budget (per run, day, month)

**Sends to** Report intake (the answer)

**Receives from** Research runs (paid API call)

`dashboard/core/providers.js:342`
`dashboard/core/providers.js:591`
`dashboard/core/providers.js:684`
`dashboard/core/providers.js:739`

## xAI Grok Outside service · Evidence services

Implemented · off until enabled · On request

One web-search call per lane each morning, made from GitHub Actions, never from the desk.

**Implemented:** Outside service; the desk's client code is at 7057e06

**Enabled:** Off by default; each desk's setting UNKNOWN

**Deployed:** Not ours to deploy

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** Repository variable SCOUT_ENABLED · Secrets XAI_API_KEY, DESK_INGEST_KEY, DESK_INGEST_URL

**Sends to** Grok Scout (answers the sweep)

`dashboard/scripts/grok-cron.js:31`
`.github/workflows/scout-cron.yml:4`

## Jupiter · KyberSwap Outside service · Evidence services

Implemented · off until enabled · On request

Keyless quote requests that answer "what would selling this size actually return?". Nothing is signed, approved or sent.

**Implemented:** Outside service; the desk's client code is at 7057e06

**Enabled:** Off by default; each desk's setting UNKNOWN

**Deployed:** Not ours to deploy

**Operating proven:** UNKNOWN: no receipt cited

**Receives from** Exit readings (quote request)

`dashboard/core/exitquotes.js:8`
`dashboard/core/exitquotes.js:12`

### Alchemy · Sourcify

Outside service · Evidence services

Implemented · off until enabled · On request

Read-only chain access for wallet history and contract state. The contract reader refuses any method outside its read allowlist.

**Implemented:** Outside service; the desk's client code is at 7057e06

**Enabled:** Off by default; each desk's setting UNKNOWN

**Deployed:** Not ours to deploy

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** ALCHEMY_API_KEY on the host

**Sends to** Wallet sync (transfers)

**Receives from** Contract forensics (chain reads)

`dashboard/core/alchemy.js:125`
`dashboard/core/contract-rpc.js:287`
`dashboard/core/contract-forensics.js:255`

### GeckoTerminal

Outside service · Evidence services

Implemented · off until enabled · On request

Closed hourly candles from one frozen pool per pair, for the Trader's Paper Book only.

**Implemented:** Outside service; the desk's client code is at 7057e06

**Enabled:** Off by default; each desk's setting UNKNOWN

**Deployed:** Not ours to deploy

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Hourly candles (closed hours)

`dashboard/core/trader-candles.js:68`

# Identify A project and its identity

## New research Eric · Identify

Implemented · on by default · Eric acts

Eric's front door for a new project. The desk finds CoinGecko candidates and opens the project and a Claude research run once one is confirmed: by Eric, or by itself when his own link or contract resolves to exactly one listing.

- Confirms itself only for a request whose CoinGecko link or exact contract resolves to one unambiguous candidate.
- A name search, a screenshot or several candidates wait for Eric's choice.
- A screenshot is read by Claude only with a key, prices and budget.
- Claude is the only model asked by default.

**Enabled:** Acts only when Eric does

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** Eric's choice, unless his own link or contract matches exactly one listing

**Checked in the code** self-confirms only a single unambiguous candidate (`intake.js:587`) · a screenshot never self-confirms (`intake.js:583`)

**Sends to** CoinGecko (searches candidates) · Project records (opens the project) · Research runs (requests Claude research)

`dashboard/core/intake.js:159`
`dashboard/core/intake.js:335`
`dashboard/core/intake.js:609`
`dashboard/core/intake.js:582`

## Grok Scout Workflow · Identify

Implemented · off until enabled · Automated

A GitHub Actions job that sends the pinned scout prompt to Grok over four lanes and posts one brief to the desk. A separate leads call names tokens.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Deployed:** Runs on GitHub from main

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** SCOUT_ENABLED for the brief · SCOUT_LEADS for token leads

**Sends to** Scout briefs (posts the brief) · Scout leads (posts token leads)

**Receives from** xAI Grok (answers the sweep)

`.github/workflows/scout-cron.yml`
`dashboard/scripts/grok-cron.js:244`
`dashboard/scripts/grok-cron.js:371`
`templates/scout/scout-prompt.v1.md`

## Scout briefs Records · Identify

Implemented · off until enabled · Automated

Briefs stored create-only on the online desk. A brief is something Eric reads; it never becomes research, a ticket or a prompt.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Receives from** Grok Scout (posts the brief)

`dashboard/core/scout.js:266`
`dashboard/core/scout.js:408`

## Scout leads Desk module · Identify

Implemented · off until enabled · Both

Token leads checked against CoinGecko's listing and the GoPlus screen under Eric's own thresholds. Only his "Confirm and request research" admits one.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** leads.enabled and Eric's thresholds · Eric confirms the rows he saw (or automatic admission, if armed)

**Sends to** CoinGecko (checks the listing) · GoPlus (checks the contract) · Automatic admission (eligible leads) · Project records (Eric's confirm admits)

**Receives from** Grok Scout (posts token leads)

`dashboard/core/leads.js:148`
`dashboard/core/leads.js:593`
`dashboard/core/leads.js:948`

## Automatic admission Desk module · Identify

Implemented · off until enabled · Automated

When Eric arms it, admits up to his daily limit of eligible leads, one pipeline at a time, leaving 15% headroom on every budget downstream.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** automation.lead_admission.enabled (false by default) · An expiry Eric sets · Blocked while automation is paused

**Checked in the code** off by default (`workspace.js:152`)

**Sends to** Project records (admits inside caps)

**Receives from** Scout leads (eligible leads)

`dashboard/core/lead-admission.js:308`
`dashboard/core/lead-admission.js:378`
`dashboard/core/workspace.js:152`

## Project records Records · Identify

Implemented · on by default · On request

One folder per project: its verified identity, research runs, reports, forensics, journal and notes. Archive is reversible; nothing is deleted.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Research runs (identity)

**Receives from** New research (opens the project) · Scout leads (Eric's confirm admits) · Automatic admission (admits inside caps)

`dashboard/core/projects.js:300`
`dashboard/core/projects.js:456` `WORKSPACE-MAP.md`

## Market prices Desk module · Identify

Implemented · on by default · Automated

Keeps the latest price, cap and volume for every tracked coin, with each figure's age.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Research runs (dated prices in the packet)

**Receives from** CoinGecko (prices) · Job timers (every minute)

`dashboard/core/market.js:483`
`dashboard/server.js:874`

## Fundamentals Desk module · Identify

● Implemented · on by default · Automated

Fees and revenue for bound projects, frozen into the research packet. A project without a binding reads “unmapped”.

**Enabled:** On by default; each desk’s setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Research runs (fundamentals in the packet)

**Receives from** DefiLlama (fees, revenue)

`dashboard/core/fundamentals.js:663`
`dashboard/core/fundamentals.js:868`

## Contract screening Desk module · Identify

● Implemented · on by default · Automated

GoPlus flags per contract, shown on the desk and carried into new research and ticket packets from the saved screen. An unanswered check stays unknown.

**Enabled:** On by default; each desk’s setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Checked in the code** carried into packets by default (`workspace.js:80`) · weekly (`workspace.js:70`)

**Sends to** Research runs (contract flags in the packet) · PM tickets (contract flags in the packet)

**Receives from** GoPlus (flags)

`dashboard/core/screening.js:660`
`dashboard/core/screening.js:914`
`dashboard/core/screening.js:53`

# Research Frozen prompts, graded reports

## Research runs Desk module · Research

Implemented · on by default · Both

A run freezes one prompt (the same bytes for every model) and one evidence packet, then keeps a job per model with careful paid-attempt bookkeeping and spending limits.

- A job goes to an API, an assistant, or a pasted answer.
- Paid attempts move reserved → dispatched → captured → responded, or uncertain.
- The research tick runs every 2 minutes.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** Spending limits · research.unattended_enabled for scheduled refreshes (off) · Nothing is sent to a provider while automation is paused

**Checked in the code** scheduled refreshes off by default (`workspace.js:118`) · the research job skips while paused (`jobs.js:267`)

**Sends to** Model APIs (paid API call) · The connector (assistant handoff) · Pilot runner (claimed work)

**Receives from** New research (requests Claude research) · Market prices (dated prices in the packet) · Fundamentals (fundamentals in the packet) · Contract screening (contract flags in the packet) · Project records (identity) · Pinned prompts (the frozen prompt) · Job timers (every 2 minutes)

`dashboard/core/research.js:1043`
`dashboard/core/research.js:873`
`dashboard/core/research.js:930`
`dashboard/core/research.js:2376`

## Pinned prompts Records · Research

Implemented · on by default · On request

Prompts, rubrics and schemas are published versions, each pinned by its SHA-256. A byte that differs from the lock is refused, and a published template is never edited.

- @6 adds the payment waterfall, cash flow per token and a falsifiable thesis.
- Thirteen category grades under the @2 scoring profiles.
- Six investment weights, cr-grading@1.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Research runs (the frozen prompt)

`dashboard/core/prompts.js:532`
`dashboard/core/template-lock.js:41`
`templates/research-prompt.v6.md`
`templates/LOCK.json`

## The connector Desk module · Research

Implemented · on by default · Both

The remote tool server the assistants sign in to with Eric's approval. Each connection is bound to its model and files only that model's work. The desk calls no model here; the assistant does the research.

- 33 tools: research, reports, portfolio, tickets, forensics, challenges.
- Tickets and forensic investigations are Claude's; challenges are ChatGPT's.
- Online desk only.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** Eric approves each connection with his passphrase

**Sends to** Report intake (save_report)

**Receives from** Research runs (assistant handoff) · Forensic audits (Claude and ChatGPT via tools) · Front door (connector)

`dashboard/mcp.js:2199` `dashboard/mcp.js:108`
`dashboard/oauth.js:177`

## Pilot runner Workflow · Research

Implemented · off until enabled · Both

Runs Agent 1 and then Agent 2 as separate Claude sessions, claiming work from the desk and handing back answers. No API key, nothing billed by the desk.

- One claim per project, one agent at a time.
- The Windows task is registered Disabled until Eric enables it.
- Claims are refused while automation is paused.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Deployed:** PC task registered at R0 on a dedicated runner at 5a8e75b, Disabled.

**Operating proven:** None recorded. At R0 the pilot-all task was Disabled with execution state never_run; B0 and later gates are unstarted.

`docs/engineering/OPERATING_PROOF_R0_2026-11-03.md:33` `docs/ONE-SHEET.md:74`

**What has to be true** Eric enrolls each project · Worker and poll token hashes on the desk · Eric enables the task

**Checked in the code** the task is registered disabled (`pilot-task.ps1:209`) · claims refused while paused (`pilot-claims.js:122`)

**Sends to** Agent 1 · Researcher (starts Agent 1) · Agent 2 · Prosecutor (starts Agent 2)

**Receives from** Research runs (claimed work)

`dashboard/scripts/pilot.js:768`
`dashboard/core/pilot-worker.js:160`
`dashboard/core/pilot-claims.js:162`
`docs/ONE-SHEET.md:74`

## Agent 1 · Researcher AI agent · Research

Implemented · on by default · Automated

Writes Claude's investment research on one project under the run's frozen prompt. It never writes tickets, never sizes, never decides.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Report intake (Claude's report)

**Receives from** Pilot runner (starts Agent 1)

`.claude/agents/forensic-researcher.md`
`.claude/commands/pilot-run.md`

## Report intake Desk module · Research

Implemented · on by default · Automated

Every report, from an API, an assistant or a paste, comes in one way: found, parsed, and bound to its run. A field that does not match the run is refused by name.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Grading (normalized report)

**Receives from** Model APIs (the answer) · The connector (save_report) · Agent 1 · Researcher (Claude's report)

`dashboard/core/inbox.js:558`
`dashboard/core/reports.js:835`
`dashboard/core/binding.js:96`

## Grading Deterministic engine · Research

● Implemented · on by default · On request

The desk calculates each report's investment grade from its six weighted components. An unknown component withholds the grade instead of guessing it. Grades are never averaged across models.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Evidence checks (graded report) · Compare & revisions (per-model grades) · PM tickets (Claude's grade is the basis) · Report performance (every eligible report)

**Receives from** Report intake (normalized report)

`dashboard/core/scoring.js:248`
`dashboard/core/scoring.js:95`

## Evidence checks Deterministic engine · Research

● Implemented · on by default · On request

Reads meaning, not field presence: does a code-quality grade name files at a commit, does a liquidity grade cite a dated depth? Flags never change a score.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Receives from** Grading (graded report)

`dashboard/core/evidence.js:491`
`dashboard/core/textscores.js:117`
`dashboard/core/consistency.js:83`

## Compare & revisions Deterministic engine · Research

● Implemented · on by default · On request

Picks each model's current report by one rule, compares numbers only under the same rubric, and labels revisions from the stored records.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Receives from** Grading (per-model grades)

`dashboard/core/compare.js:24`
`dashboard/core/compare.js:51`
`dashboard/core/revisions.js:69`

# Challenge The short case and the exit

## Forensic audits Desk module · Challenge

● Implemented · on by default · Both

A separate investigation of whether a project is impaired, degrading, inactive or discontinued. It changes no grade. Receipts are create-only, with fixed checkpoints from 72 hours to 120 days.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** The connector (Claude and ChatGPT via tools)

`dashboard/core/forensics.js:569`
`dashboard/core/forensics.js:666`
`dashboard/core/forensics.js:841`
`templates/forensics/forensic-prompt.v1.md`

## Forensic monitor Desk module · Challenge

○ Implemented · off until enabled · Automated

For projects Eric opts in: guarded product checks, GitHub reads, exit quotes, and the day's checkpoint resolutions. Never a model call.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** Eric turns monitoring on per project · Eric approves the chains to quote

**Sends to** Exit readings (due quotes)

**Receives from** Job timers (every 30 minutes)

`dashboard/core/forensic-monitor.js:365`
`dashboard/core/netguard.js:153`
`dashboard/core/forensics.js:1016`

## Exit readings Deterministic engine · Challenge

○ Implemented · off until enabled · Automated

Quotes the exit at the position's size every 12 hours, for projects whose quotes Eric approved (or his standing policy, if he turned it on). A cost is complete only when the network fee and the transfer tax are known; otherwise it reads "at least".

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** Eric approves a chain per project, or exit_quotes_auto (off by default) · The standing policy is blocked while automation is paused

**Checked in the code** the standing policy is off by default (`workspace.js:141`)

**Sends to** Jupiter · KyberSwap (quote request) · PM tickets (exit readings in the packet)

**Receives from** Forensic monitor (due quotes)

`dashboard/core/exitquotes.js:385`
`dashboard/core/forensics.js:1199`

## Contract forensics Deterministic engine · Challenge

○ Implemented · off until enabled · Automated

Reads a contract at a finalized block: code hash, proxy, controllers, supply and authority probes, then a rule-based verdict. It only affects sizing when the gate is set to enforce; by default it observes.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** contracts.enabled and compute caps (off by default) · tickets.contract_gate.mode = enforce (observe by default)

**Checked in the code** contract reads off by default (`workspace.js:193`)

**Sends to** Alchemy · Sourcify (chain reads) · PM tickets (contract verdict)

`dashboard/core/contract-forensics.js:1264`
`dashboard/core/contract-gate.js:279`
`dashboard/core/workspace.js:173`

### Agent 2 · Prosecutor AI agent · Challenge

Implemented · on by default · Automated

Attacks Claude's current report as a short-seller and writes one schema-checked ticket with a trade plan: entry zone, upside target, invalidation. It never chooses a dollar amount.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** PM tickets (the ticket)

**Receives from** Pilot runner (starts Agent 2) · PM tickets (the frozen packet)

`.claude/agents/junior-trader-prosecutor.md`
`templates/tickets/ticket-prompt.v1.md`
`templates/schemas/ticket.v1.schema.json`

### ChatGPT challenge AI agent · Challenge

Implemented · on by default · Both

ChatGPT may check a saved ticket on four tests. Its verdict shows on the decision card in words and changes no ticket, gate, size or book.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Eric's decision (advice in words)

**Receives from** PM tickets (a saved ticket)

`dashboard/core/ticket-challenges.js:240`
`dashboard/core/ticket-challenges.js:356`
`templates/tickets/challenge-prompt.v1.md`

# Authorize A ceiling, then Eric or his mandate

## PM tickets Desk module · Authorize

Implemented · on by default · Automated

Opens a ticket from Claude's current report, freezes the evidence packet and the Prosecutor's prompt, and saves the Prosecutor's ticket as a receipt that never changes.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Agent 2 · Prosecutor (the frozen packet) · ChatGPT challenge (a saved ticket) · Sizing gate (grade, exits, held)

**Receives from** Contract screening (contract flags in the packet) · Grading (Claude's grade is the basis) · Exit readings (exit readings in the packet) · Contract forensics (contract verdict) · Agent 2 · Prosecutor (the ticket)

`dashboard/core/tickets.js:669`
`dashboard/core/tickets.js:1387`
`dashboard/core/tickets.js:2784`

## Sizing gate Deterministic engine · Authorize

Implemented · on by default · Automated

Fixed rules turn the grade, the exit reading and whether Eric holds the asset into an allowed ticket type and a suggested risk: $500 standard, $250 starter, $0 below the floor. The suggested risk is a ceiling on what a grant may risk, not a position size, an order or a fill.

- Below 48 on an asset Eric does not hold, no Prosecutor runs (pregate).
- A ticket whose invalidation cannot be checked against a fresh price is held to $0.
- A trigger already breached at save writes nothing.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Checked in the code** $500 standard (`tickets.js:110`) · $250 starter (`tickets.js:111`) · pregate at 48 (`tickets.js:113`)

**Sends to** Eric's decision (suggested risk ceiling) · Paper mandate (ceiling for a grant)

**Receives from** PM tickets (grade, exits, held)

`dashboard/core/tickets.js:441`
`dashboard/core/tickets.js:535`
`dashboard/core/tickets.js:1305`

## Eric's decision Eric · Authorize

Implemented · on by default · Eric acts

The PM Decision Card. Eric approves the suggested risk ceiling, halves it, vetoes or passes. The desk refuses a stale decision: an expired exit reading, a newer report, an invalidated trigger or a changed card.

- Approve and Halve need an exit reading under 72 hours old.
- The card names what changed when it refuses.
- A grant is a permission up to a dollar ceiling. It is not an order and not a fill.

**Enabled:** Acts only when Eric does

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** Eric alone

**Checked in the code** the exit-age refusal exists (`tickets.js:1838`) · the window is the liquidity freshness (`tickets.js:1740`) · which is 72 hours (`forensic-defs.js:63`)

**Sends to** Decision basis (frozen in the same write) · PM benchmark (Eric's grants)

**Receives from** ChatGPT challenge (advice in words) · Sizing gate (suggested risk ceiling)

`dashboard/core/tickets.js:1870`
`dashboard/core/tickets.js:1838`
`dashboard/web/src/views/inspector/PMDecisionCard.jsx`

## Paper mandate Desk module · Authorize

Implemented · off until enabled · Automated

Off by default. When Eric arms it, grants on pilot projects by grade (50–54 a halve, 55 and up an approve) on the Trader's book only, within his caps and an expiry at most 30 days ahead. Its existence in the code authorizes nothing: only an armed, unrevoked authority generation does.

- A pause, or switching the loop off, revokes the generation; a resume does not re-arm it.
- Any decision of Eric's own on a project ends the mandate there.
- Mandate grants never enter the PM benchmark.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** None recorded: no mandate grant is cited by the repository's operating receipts. `docs/ONE-SHEET.md:74`

**What has to be true** automation.autonomous_paper_loop (false by default) · An authority generation recorded when Eric arms it · An expiry within 30 days · Not paused

**Checked in the code** off by default (`workspace.js:142`) · grants from grade 50 (`tickets.js:1572`) · expiry at most 30 days (`tickets.js:1573`) · never enters the PM benchmark (`tickets.js:1565`) · a pause revokes the generation (`mandate-authority.js:290`)

**Sends to** Decision basis (grants inside limits, if armed)

**Receives from** Sizing gate (ceiling for a grant) · Off switches (stops)

`dashboard/core/tickets.js:2667`
`dashboard/core/mandate-authority.js:165`
`dashboard/core/mandate-authority.js:277`
`dashboard/core/workspace.js:134`

## Off switches Interface · Authorize

Implemented · on by default · Eric acts

One page of off-switches. A project stop halts admission, the pilot, mandate grants and openings for that project. The desk-wide pause stops new automated dispatches; settlement, candles, wallet sync and reconciliation go on.

**Enabled:** Acts only when Eric does

**Operating proven:** UNKNOWN: no receipt cited

**Checked in the code** the pause leaves settlement and reconciliation running (`workspace.js:131`)

**Sends to** Paper mandate (stops)

`dashboard/core/desk-controls.js:77`
`dashboard/core/mandate-authority.js:176`
`dashboard/core/workspace.js:129`

# Record Decision basis and four books

## Decision basis Records · Record

● Implemented · on by default · Automated

Freezes the facts behind every decision in the same write as the decision, with a SHA-256 ledger checked on every read.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Trader adapter (approve or halve)

**Receives from** Eric's decision (frozen in the same write) · Paper mandate (grants inside limits, if armed)

`dashboard/core/decision-basis.js:147`
`dashboard/core/decision-basis.js:285`

## PM benchmark Records · Record

● Implemented · on by default · Automated

One paper position per approve or halve of Eric's, voided on Undo, compared with BTC. Its result is the granted dollars times the price return: decision attribution, not funded execution. No totals before 20 closed positions. Mandate grants never enter it.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Checked in the code** result = granted dollars × return (`paper.js:299`)

**Receives from** Eric's decision (Eric's grants)

`dashboard/core/paper.js:150`
`dashboard/core/paper.js:202`
`dashboard/core/paper.js:402`

## Trader adapter Deterministic engine · Record

○ Implemented · off until enabled · Automated

The only caller of the Trader's book. Turns an approve or halve (or a mandate grant) into one trade intent: entry, stop, target, and a planned loss at the stop of at most the grant. The quantity, and so the position notional, follow from that, under the notional cap. It cancels what has not filled when a mandate no longer covers a buy, and closes on Undo, the horizon's end or an invalidation.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** None recorded: no funded book or intent is cited by the repository's operating receipts.

`docs/ONE-SHEET.md:74`

**What has to be true** trader.book funding and fee (no default) · trader.rules: stop distance, notional, allowlist

**Checked in the code** the intent's risk is the granted amount (`trader-desk.js:390`) · notional is computed separately (`trader-desk.js:589`)

**Sends to** Risk engine (intent)

**Receives from** Decision basis (approve or halve) · Hourly candles (fills, stops, targets) · Job timers (every 30 minutes)

`dashboard/core/trader-desk.js:781`
`dashboard/core/trader-desk.js:818`
`dashboard/core/trader-desk.js:264`
`dashboard/core/trader-desk.js:610`

### Risk engine Deterministic engine · Record

Implemented · on by default · On request

Checks every intent before it is sent: 2% of the book per trade, 4% drawdown in 24 hours, 20 bps slippage, the allowlist, the stop distance, the notional and exit caps, and a halt while any order is unresolved.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Checked in the code** 2% per trade (`trade-risk.js:65`) · 4% drawdown (`trade-risk.js:66`) · 20 bps (`trade-risk.js:67`)

**Sends to** Trader simulation (checked intent)

**Receives from** Trader adapter (intent)

`dashboard/core/trade-risk.js:135`
`dashboard/tests/trader-boundary.test.js:30`

### Trader simulation Records · Record

Implemented · off until enabled · Automated

A simulated book with a hash-chained journal. Its "venue" is a file. An order is accepted, refused, waits, fills in part or in full; it never times out to lost, and stays unknown until the venue is asked.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** None recorded: no simulated order or fill is cited by the repository's operating receipts.

`docs/ONE-SHEET.md:74`

**Sends to** Report performance (book results)

**Receives from** Risk engine (checked intent)

`dashboard/core/paper-trader.js:867`
`dashboard/core/paper-trader.js:827`
`dashboard/core/paper-trader.js:927`
`dashboard/core/paper-trader.js:1077`

### Hourly candles Desk module · Record

Implemented · off until enabled · Automated

Closed hourly candles for each pair. A missing hour is never filled in. Asks nothing until the book is funded.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Trader adapter (fills, stops, targets)

**Receives from** GeckoTerminal (closed hours)

`dashboard/core/trader-candles.js:522`
`dashboard/core/trader-candles.js:186`

### Report performance Deterministic engine · Record

Implemented · on by default · On request

Each eligible report's return against BTC at 30, 60, 90 and 120 days, beside its own verdict, traded or not. Never a ranking, never "most accurate", nothing stored.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Receives from** Grading (every eligible report) · Trader simulation (book results)

`dashboard/core/report-outcomes.js:220`
`dashboard/core/grade-outcome-view.js:250`

## Real holdings Records · Record

● Implemented · on by default · Eric acts

Eric's own ledger of real holdings and cost, kept apart from every paper book. Nothing here places a trade.

**Enabled:** Acts only when Eric does

**Operating proven:** UNKNOWN: no receipt cited

**Receives from** Wallet sync (verified events)

`dashboard/core/portfolio.js:222`
`dashboard/core/portfolio.js:490`

## Wallet sync Desk module · Record

○ Implemented · off until enabled · Both

Follows public addresses Eric adds, twice a day. Only final, verified events become ledger entries; a possible swap waits for his decision.

**Enabled:** Off by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**What has to be true** ALCHEMY_API_KEY · A wallet Eric adds himself

**Checked in the code** twice a day (`workspace.js:171`)

**Sends to** Real holdings (verified events)

**Receives from** Alchemy · Sourcify (transfers)

`dashboard/core/walletsync.js:506`
`dashboard/core/wallets.js:1121`
`dashboard/core/wallets.js:266`

# Platform Where it runs and how it keeps records

## Online desk Desk module · Platform

Implemented · on by default · Automated

The same code, deployed from main to Render, with its own records on a Render disk. It refuses to start without a disk, a passphrase and a host name.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** Serving: /healthz read back 5a8e75b at 2026-11-03 17:20 UTC (R0). That proves the revision served, not that any job on it ran correctly.

`docs/engineering/OPERATING_PROOF_R0_2026-11-03.md:23`

**Sends to** Front door (every request)

**Receives from** Deploy (deploys main)

`server.js` `dashboard/hosted.js:466` `dashboard/hosted.js:174`

## PC desk Desk module · Platform

Implemented · on by default · Both

The desk on Eric's PC, bound to this machine only. Its hourly Windows task, when enabled, runs due jobs and pulls the only off-Render copy of the records.

**Enabled:** On by default; each desk's setting UNKNOWN

**Deployed:** At R0 the PC tasks run from a dedicated runner checkout at 5a8e75b; the local desk's own revision is not recorded.

**Operating proven:** Not current: at R0 Hourly was left temporarily Disabled, so no new offsite run is claimed.

`docs/engineering/OPERATING_PROOF_R0_2026-11-03.md:89`

`dashboard/core/schedule.js:31` `dashboard/scripts/scheduled-run.cmd` `docs/engineering/OPERATING_PROOF_R0_2026-11-03.md:88`

## Front door Desk module · Platform

Implemented · on by default · Automated

Everything a request meets before the desk: security headers, the health check, host checks, the pilot and connector routes, scout ingest, then the passphrase sign-in.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Desk API (signed in) · The connector (connector)

**Receives from** Online desk (every request)

`dashboard/frontdoor.js:125` `dashboard/hosted.js:337`

## Desk API Desk module · Platform

Implemented · on by default · On request

The signed-in API behind the app. Every call carries a per-launch token.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Journaled writes (changes)

**Receives from** Front door (signed in) · Desk app (reads and actions)

`dashboard/server.js:975` `dashboard/server.js:524`

### Job timers Desk module · Platform

Implemented · on by default · Automated

Market every minute, research every 2 minutes, and every 30 minutes: snapshot, reviews, backup, fundamentals, screening, forensics, trader, leads. A job on a timer still checks its own gates each time; a timer is not proof that a job ran.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Checked in the code** the hosted timer list exists (`jobs.js:41`)

**Sends to** Market prices (every minute) · Research runs (every 2 minutes) · Forensic monitor (every 30 minutes) · Trader adapter (every 30 minutes)

`dashboard/core/jobs.js:41`
`dashboard/core/jobs.js:294`

### Desk app Interface · Platform

Implemented · on by default · Eric acts

Desk, Review, Portfolio, Research and Outcomes, plus the inspector and each project's page. Opening a page never writes.

**Enabled:** Acts only when Eric does

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Desk API (reads and actions)

`dashboard/web/src/lib/router.js`
`dashboard/web/src/views/watchlist/StatusStrip.jsx`

### Journaled writes Deterministic engine · Platform

Implemented · on by default · On request

Record changes are designed to go through one operation: lock, stage, atomic move, verify, log. A save counts only when it reads back verified. Not audited here as universal: some modules (captures, exports, leases, pilot claims) also write files directly.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Workspace files (atomic writes)

**Receives from** Desk API (changes) · CLI (changes) · Request queue (changes)

`dashboard/core/ops.js:665`
`dashboard/core/ops.js:409`
`dashboard/core/fsx.js:49`

### Workspace files Records · Platform

Implemented · on by default · On request

No hidden database. Projects, portfolio, market data, inbox, logs and backups are folders of files.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Backups (daily)

**Receives from** Journaled writes (atomic writes)

`WORKSPACE-MAP.md` `dashboard/core/workspace.js:355`

### Backups Desk module · Platform

Implemented · on by default · Automated

A sealed daily backup on the desk's disk, and an offsite package the PC pulls with its own key. A restored copy asks no provider anything until released.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Receives from** Workspace files (daily)

`dashboard/core/exporter.js:826`
`dashboard/core/offsite.js:197`
`dashboard/core/offsite.js:310`

### CLI Interface · Platform

Implemented · on by default · Eric acts

Every record change from a terminal: doctor, research, tickets, forensics, paper, wallets, backups, tests.

**Enabled:** Acts only when Eric does

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Journaled writes (changes)

`dashboard/cli.js`

### Research Explorer Desk module · Platform

Implemented · off until enabled · Eric acts

An optional one-way mirror into Supabase. Fictional fixtures go by command; one real project goes only by its preview's hash from Eric's own terminal; the whole desk stays refused.

**Enabled:** Acts only when Eric does

**Operating proven:** UNKNOWN: no receipt cited

**Checked in the code** a send that does not match its preview is refused (`explorer.js:605`)

`dashboard/core/explorer.js:565`
`dashboard/core/explorer.js:596`
`dashboard/core/explorer.js:515`

### Request queue Records · Platform

Implemented · on by default · Both

A session that cannot run the CLI leaves a request file; the desk processes it and writes a receipt.

**Enabled:** On by default; each desk's setting UNKNOWN

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Journaled writes (changes)

`dashboard/core/requests.js:280`

# Engineering How the code changes

## Supervised loop Eric · Engineering

● Implemented · on by default · Eric acts

One task at a time on its own branch in Eric's Claude Code session, the test suite, a pull request, and Eric's own merge.

**Enabled:** Acts only when Eric does

**Deployed:** Runs on GitHub from main

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** The test check (pull request)

`docs/CONSTITUTION.md:18`

## The test check Workflow · Engineering

● Implemented · on by default · Automated

GitHub runs the test suite on every pull request; it is the only check main requires.

**Enabled:** On by default; each desk's setting UNKNOWN

**Deployed:** Runs on GitHub from main

**Operating proven:** R0 records a full local run at the deployed revision: 1,745 tests, 1,743 passed, 0 failed, 2 opt-in probes skipped (2026-11-03). That is the suite passing, not the desk operating. `docs/engineering/OPERATING_PROOF_R0_2026-11-03.md:62`

**Sends to** Protected trader files (boundary test) · Deploy (Eric merges)

**Receives from** Supervised loop (pull request)

`.github/workflows/test.yml` `dashboard/cli.js:238`

## Protected trader files Deterministic engine · Engineering

● Implemented · on by default · Automated

Nine files (the risk engine, both paper books, the decimal maths, their tests and the intent schemas) are on one canonical list. The boundary test checks that each exists, that nothing but the desk's adapter calls the trader, that no trader module reaches a network, process or secret, and that schema and engine agree.

- It does not detect or block a change to a listed file: that rests on review of a pull request marked [P] and Eric's merge.
- A test cannot establish human intent.

**Enabled:** On by default; each desk's setting UNKNOWN

**Deployed:** A practice, not a service

**Operating proven:** UNKNOWN: no receipt cited

**Checked in the code** the canonical list names the core files (`trader-boundary.test.js:35`)

**Receives from** The test check (boundary test)

`dashboard/tests/trader-boundary.test.js:30`
`dashboard/tests/trader-boundary.test.js:31`

## Deploy Workflow · Engineering

● Implemented · on by default · Automated

Render deploys main. A merge counts as deployed only once the health check shows its commit, read back at a stated time.

**Enabled:** On by default; each desk's setting UNKNOWN

**Deployed:** A practice, not a service

**Operating proven:** UNKNOWN: no receipt cited

**Sends to** Online desk (deploys main)

**Receives from** The test check (Eric merges)

`docs/ONE-SHEET.md:73`

### Ops-1 runner Workflow · Engineering

◌ Not created · Automated

A planned hourly Render Cron Job to run the pilot after the PC's supervised proof. A specification and a preflight script exist; the service itself has not been created.

**Implemented:** Not built: a specification and preflight exist

**Enabled:** —

**Deployed:** Not created

**Operating proven:** —

`docs/engineering/OPS1_RENDER_CRON.md`
`dashboard/scripts/ops1-preflight.js`

### Agent lane Workflow · Engineering

— Retired · Automated

The self-building agent lane, the PM builder and the engineering connector were deleted on purpose. Nothing in the repository builds, reviews or merges on a schedule.

**Implemented:** Removed on purpose

**Enabled:** —

**Deployed:** Removed

**Operating proven:** —

`docs/CONSTITUTION.md:15`
`.github/rulesets/README.md`

# V · How to read this atlas

Every component, connection and claim was read from the repository's code and documents. Each source link opens the exact line on GitHub, pinned to commit `7057e06`, so it keeps pointing at what was read even as the code moves on.

The build reads every file as it is at that commit. It checks that each cited function, constant or line exists (199 source links), and that each claim the words depend on, a default, a limit or a refusal, matches the code (45 claims). It refuses to produce the atlas if any fails, if a connection names a part that does not exist, or if a part of the five stages belongs to none or to two. A matching claim proves the code says it; it does not prove the system ran.

## Four status facts

| Fact | Means | What this atlas used |
|---|---|---|
| **Implemented** | Code exists at the baseline. | Read from the repository at 7057e06; the build checks each cited line and claim. |
| **Enabled** | Configuration permits it to act. | Only the default in code is given. Each desk's current settings are not read: UNKNOWN. |
| **Deployed** | The named revision is serving. | The last receipt, R0: the online desk served 5a8e75b at 2026-11-03 17:20 UTC, older than this baseline. Current revision: UNKNOWN. The PC desk: UNKNOWN. |
| **Operating proven** | The workflow completed correctly. | Given only where a repository receipt says so; otherwise UNKNOWN or "none recorded". |

## The marks on the map

| | |
|---|---|
| ● **Implemented · on by default** | In the code at the baseline and on unless a setting turns it off. Not proof that it is deployed or ran. |
| ○ **Implemented · off until enabled** | In the code at the baseline; does nothing until a key, a setting or Eric's switch enables it. |

| | |
|---|---|
| ◌ **Not created** | Specified in a document, with some supporting code; the thing itself does not exist. |
| — **Retired** | Removed on purpose; must not be rebuilt. |
| **Automated** | An execution mode: runs on a timer or by itself once enabled. It does not prove a timer fired or that the result is healthy. |
| **Eric acts** | Happens only by Eric’s own click, command or merge |
| **Both** | Eric starts or enables it; the desk carries it out |
| **On request** | Runs only when another part calls it |

## What this atlas could not establish

- **Every record change uses the one locked path** Captures, exports, leases and pilot claims also write files directly. Not audited.
- **What each desk is set to now** The atlas reads defaults from code, never a desk’s settings.
- **Whether timers fired and their jobs succeeded** No job receipts are read.
- **The PC desk’s served revision** R0 records the runner checkout, not the local desk.
- **Whether source links resolve on GitHub** Lines are resolved locally at the baseline; GitHub is not fetched.